

알고리즘 성능 분석 - 실습

자료구조, 2026-1학기

김효승 (Hyoseung_kim@hallym.ac.kr)

정보과학대학 소프트웨어학부



- 목차

- 미니 퀴즈 풀이 (홀수 문제)
- n 까지 숫자 합 구현 ($O(1)$, $O(n)$, $O(n^2)$)
- 과제1 : 미니 퀴즈 짝수 문제
- 과제2 : 배열의 원소 합을 구하는 3가지 메소드 구현
- 과제3(추가점수): 과제2 메소드의 시간 복잡도 계산

• 아래 식의 참 / 거짓을 구분하시오

1. $n^2 = O(n^2)$

2. $n = \Omega(n^2)$

3. $n^2 = O(n^3)$

4. $2n^2 - 100n = \Omega(n^2)$

5. $3n^2 + \log n = O(n^2)$

6. $n^2 = O(n^{100})$

7. $7 \log n = O(n)$

8. $5n^2 + 7n - 3 = \Theta(n^2)$

9. $5n^2 + 7n - 3 = \Omega(n \log n)$

10. $7n - 3 = \Theta(n^2)$

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

• 아래 식의 참 / 거짓을 구분하시오

1. $n^2 = O(n^2)$

답: (T) $n_0 = 1, c = 1: n \geq 1$ 일 때 $n^2 \leq n^2$ 성립

3. $n^2 = O(n^3)$

답: (T) $n_0 = 1, c = 1: n \geq 1$ 일 때 $n^2 \leq n^3$ 성립

5. $3n^2 + \log n = O(n^2)$

답: (T) $n_0 = 1, c = 4: n \geq 1$ 일 때 $3n^2 + \log n \leq 4n^2$ 성립

7. $7 \log n = O(n)$

답: (T) $n_0 = 1, c = 7: n \geq 1$ 일 때 $7 \log n \leq 7n$ 성립

9. $5n^2 + 7n - 3 = \Omega(n \log n)$

답: (T) $n_0 = 1, c = 1: n \geq n_0$ 일 때 $5n^2 + 7n - 3 \geq n \log n$ 성립

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

• 1부터 n까지 합을 구하는 메소드 구현

- $O(1)$, $O(n)$, $O(n^2)$ 시간 복잡도를 갖는 메소드를 각각 구현
- n을 변경하여 테스트 해볼 것
 - System.nanoTime()

```
/**
 * O(1) 시간 복잡도를 가지는 메소드 구현
 * 메소드는 1부터 n까지의 합을 반환
 * 수학적 공식을 활용하여 구현
 *
 * @param n 입력값
 * @return 1부터 n까지의 합
 */
public static long sumO1(int n) {
    // 여기에 코드 작성
}
```

```
/**
 * O(n) 시간 복잡도를 가지는 메소드
 * 메소드는 1부터 n까지의 합을 반환
 * 단순 반복문을 사용하여 합을 계산
 *
 * @param n 입력값
 * @return 1부터 n까지의 합
 */
public static long sumOn(int n) {
    // 여기에 코드 작성
}
```

```
/**
 * O(n^2) 시간 복잡도를 가지는 메소드를 구현
 * 메소드는 1부터 n까지의 합을 반환
 * 중첩 반복문을 사용하여 각 숫자를 1부터 i까지 더하는 방식으로 합을 계산
 *
 * @param n 입력값
 * @return 1부터 n까지의 합
 */
public static long sumOn2(int n) {
    // 여기에 코드 작성
}
```

```
n = 10000
O(n): 50005000
실행 시간: 1923200 ns
O(n^2): 50005000
실행 시간: 13192900 ns
O(1): 50005000
실행 시간: 78000 ns
```

과제

- (과제1) 스마트리드에 리포트 형식으로 문서 작성하여 제출
- (과제2) 실습 시간 내 완성 시 확인 받고 퇴실 가능
- (과제3) 과제2의 시간 복잡도 분석한 리포트 제출 // *optional*

- 아래 식의 참 / 거짓을 구분하시오

2. $n = \Omega(n^2)$

4. $2n^2 - 100n = \Omega(n^2)$

6. $n^2 = O(n^{100})$

8. $5n^2 + 7n - 3 = \Theta(n^2)$

10. $7n - 3 = \Theta(n^2)$



- 배열 원소의 합 구하기

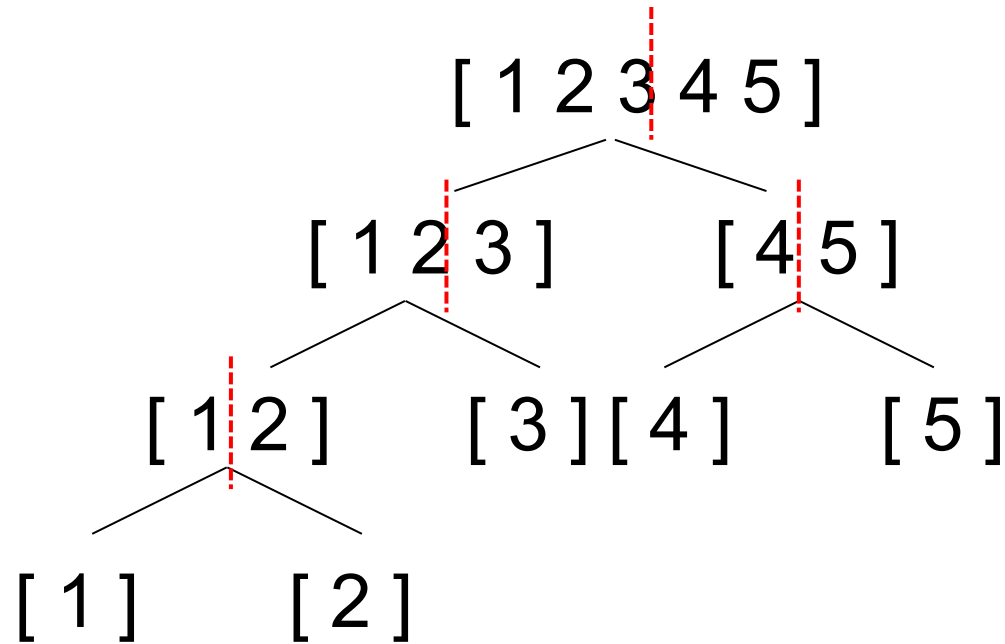
- 3가지 방법으로 메소드 구현
 - 반복문 사용, 단순 재귀 사용, 분할-정복 방법 사용

```
/**
 * 반복문을 사용하여 배열의 모든 원소의 합을 구함
 * 배열을 한 번 순회하면서, 각 원소의 값을 누적하여 합을 구함
 *
 * @param arr 정수 배열
 * @return 모든 요소의 합
 */
public static int sumWithLoop(int[] arr) {
    // 여기에 코드를 작성
}
```

```
/**
 * 재귀를 사용하여 배열의 모든 원소의 합을 구함
 *
 * @param arr 정수 배열
 * @param index 현재 요소의 인덱스
 * @return 모든 요소의 합
 */
public static int sumWithRecursion(int[] arr, int index) {
    // 여기에 코드를 작성
}
```


- 배열 원소의 합 구하기

- 3가지 방법으로 메소드 구현
 - 반복문 사용, 재귀 사용, 분할-정복 방법 사용



(재귀적 구조)

- 배열 분할
- 왼쪽 배열 합과 오른쪽 배열 합을 합산

- 배열 원소의 합 구하기

- 3가지 방법으로 메소드 구현
 - 반복문 사용, 재귀 사용, 분할-정복 방법 사용

```
/**
 * 배열을 분할 정복하는 방식으로 모든 원소의 합을 구함
 * 배열을 반으로 나누고, 각 부분의 합을 재귀적으로 계산한 다음, 두 합을 구함
 *
 * @param arr 정수 배열
 * @param start 시작 인덱스
 * @param end 끝 인덱스
 * @return 모든 요소의 합
 */
public static int sumWithDivideAndConquer(int[] arr, int start, int end) {
    // 여기에 코드를 작성
    // 시작 인덱스와 끝 인덱스가 같으면 (= 나눈 배열의 크기가 1이면) 해당 원소를 반환

    // 중간 지점 mid를 계산

    // mid를 기준으로 왼쪽 부분의 합을 변수로 지정

    // mid를 기준으로 오른쪽 부분의 합을 변수로 지정

    // 왼쪽 합과 오른쪽 합을 더하여 반환
}
```



Thank You

Q&A